

PRODUCTION READINESS

Deep Security & Correctness Audit

rust-hft-arb | Cross-Exchange Arbitrage System

77 source files | 17 exchange implementations | ~25,000 lines of Rust

Audit Date: 2026-07-18

1. Executive Summary

VERDICT: NOT PRODUCTION READY. This system has **35 CRITICAL**, **66 HIGH**, **90+ MEDIUM**, and **40+ LOW** findings across 77 audited source files. The system will lose real money if deployed in its current state. Several findings represent direct capital-loss vectors with no safety net, while others indicate fundamental architectural gaps where safety-critical code exists but is never wired into the production execution path.

The audit examined every line of code across seven subsystems: core execution pipeline, risk and protection systems, balance and capital management, data feed and orderbook handling, all 17 exchange implementations, infrastructure and support modules, and P&L; reporting. Each subsystem was evaluated for correctness, safety, resilience, concurrency, performance, and financial safety. The findings are categorized by severity and mapped to specific code locations with concrete live-trading impact scenarios and suggested fixes.

The most dangerous category of findings is the three safety-critical modules (SafetyExecutionEngine, CoreExecutionShield, CrossExchangeExecutor) that exist in the codebase but are **never called** from the production execution path. This means there is no pre-trade price validation, no circuit breaker integration, and no quantity mismatch detection in the live trading loop. Orders flow directly from strategy signals to exchange APIs with no safety gate. Additionally, the withdrawal system has no address whitelist, no amount caps, and no confirmation step, creating a direct fund-theft vector. The L2 orderbook feed does not invalidate stale data on WebSocket disconnect, and the circuit breaker auto-recovers from network partitions after 60 seconds, which is the number-one scenario for catastrophic loss in distributed trading systems.

Severity	Count	Definition
CRITICAL	35	Immediate capital loss or system breakage
HIGH	66	Significant correctness or safety concern
MEDIUM	90+	Correctness under specific conditions
LOW	40+	Code quality / maintainability

The report is organized by subsystem. Within each section, findings are listed in severity order. The Top 10 Priority Fixes section at the end provides a ranked remediation roadmap. Every finding includes the exact file and line number, a description of the issue, a concrete live-trading impact scenario, and a suggested fix.

2. Core Execution Pipeline

The core execution pipeline is the heart of the trading system, responsible for receiving signals, sizing orders, dispatching them to exchanges, and handling fills. The audit examined `main.rs`, `execution.rs`, `strategies.rs`, `cross_exchange_executor.rs`, `safety_execution.rs`, and `core_execution_shield.rs`. This subsystem contains the highest concentration of critical findings because it controls the actual flow of money.

2.1 Critical Findings

Severity	ID	Finding
CRITICAL	X-01	SafetyExecutionEngine, CoreExecutionShield, and CrossExchangeExecutor are all dead code. None are wired into the production path. There is NO pre-trade price validation, NO circuit breaker, NO quantity mismatch detection, and NO order type enforcement in live trading.
CRITICAL	E-01	blast_arbitrage_legs never deducts filled amounts from LocalCapitalAllocator. After each trade, the balance matrix becomes stale, leading to oversized orders and exchange rejections.
CRITICAL	E-03	Asymmetric fill detection uses XOR (a_filled ^ b_filled) which only catches one-side-complete failures. It misses quantity mismatch (both fill but different amounts), creating naked positions.
CRITICAL	E-02	Counter-order P&L; loss is never recorded to the daily loss counter. Untracked losses accumulate and the daily loss limit never triggers, allowing unlimited losses from rollbacks.
CRITICAL	M-01	Triangular arb fires three independent USDT pairs instead of a proper closed loop. This guarantees losses equal to 3x taker fees on every triangular execution.
CRITICAL	M-02	force_live_mode silently falls back to PaperExchangeClient on exchange init failure. The operator thinks they are live, but some legs execute on paper while others execute with real money.
CRITICAL	SE-01	SafetyExecutionEngine exists but is never called. No pre-trade slippage guard, no market order prohibition, no unique client_order_id generation in the live path.
CRITICAL	CS-01	CoreExecutionShield provides circuit breaker, depth simulation, and fee-aware profit validation, but is never wired into the signal-to-execution path.

2.2 High Findings

Severity	ID	Finding
HIGH	E-05	Slippage verification is advisory only. verify_leg_slippage returns Err but the caller ignores it. No counter-order is triggered for excessive slippage.
HIGH	E-04	Total timeout (500ms) makes the 3-retry mechanism useless. A single leg worst-case needs 1200ms but the total timeout fires at 500ms.
HIGH	S-02	Fee schedule try_read() failure silently skips fee deduction. Signals evaluated without fee deduction during dynamic fee updates.
HIGH	S-01	Triangular profit ratio has up to 3 bps truncation error from three sequential integer divisions, causing unprofitable trades to appear profitable.
HIGH	E-08	Counter-order uses 50 bps nudge (10x the normal 5 bps slippage tolerance) with losses never recorded.
HIGH	M-04	decimal_to_fp uses fragile string-based conversion that can fail on scientific notation. Returns 0 on failure.

3. Risk and Protection Systems

The risk and protection subsystem is the last line of defense against catastrophic loss. It implements a 14-layer pre-trade gatekeeper (protections.rs), a VWAP slippage engine (production_risk_shield.rs), depeg protection (depeg_protection.rs), circuit breaker (circuit_breaker.rs), volatility guard (volatility_guard.rs), and capital starvation detector (capital_starvation.rs). While the architecture is well-designed in concept, the implementation has several critical gaps that allow risk limits to be bypassed.

3.1 Critical Findings

Severity	ID	Finding
CRITICAL	R-C01	No exposure release mechanism in <code>protections.rs</code> . <code>try_reserve_exposure</code> adds to <code>total_exposure</code> but there is no <code>release_exposure</code> . Exposure monotonically increases until the 50% cap permanently saturates.
CRITICAL	R-C02	<code>DrawdownTracker</code> initialized with <code>equity=0</code> . Layer 6 (<code>MaxDrawdown</code>) can never fire until <code>set_initial_equity</code> is called. If boot sequence forgets, drawdown protection is permanently disabled.
CRITICAL	R-C03	TOCTOU race in session PnL checks (Layers 4 and 5). Multiple concurrent trades can each pass the PnL check individually but combined losses exceed the hard cap.
CRITICAL	R-C04	VWAP approximation after lot truncation is fundamentally wrong. Proportional scaling overestimates cost, rejecting profitable trades or producing incorrect estimates.
CRITICAL	R-C05	Circuit breaker auto-recovers from <code>NETWORK_PARTITION</code> and <code>CLOCK_DRIFT</code> after 60 seconds. After a network partition, trading resumes on stale state, potentially doubling positions.

3.2 High Findings

Severity	ID	Finding
HIGH	R-H08	<code>attempt_recovery()</code> in <code>depeg_protection</code> bypasses the 10-tick hysteresis. A single price flicker can resume trading during an ongoing depeg.
HIGH	R-H12	Volatility guard acquires two mutexes on every <code>check_spread()</code> call. Lock contention on the HFT hot path causes latency spikes.
HIGH	R-H14	Capital starvation tracks exchange IDs only, not (exchange, token) pairs. USDT starvation on one exchange blocks ALL trades on that exchange.
HIGH	R-H10	Circuit breaker <code>reset()</code> is public with no access control. Any code path can unfreeze trading, even after <code>BALANCE_CORRUPTION</code> .
HIGH	R-H04	<code>Utc::now()</code> system call on every hot-path check. NTP clock skew can cause stale data to appear fresh, bypassing time-based protections.

4. Balance, Capital, and Rebalancing

The balance and capital subsystem manages fund allocation across exchanges, tracks available trading capital, and handles inter-exchange transfers. Financial calculations correctly use `rust_decimal` (no f64 usage found). However, the atomic operations for balance tracking have a critical underflow vulnerability, and the rebalancer has a dangerous deposit verification bug.

4.1 Critical Findings

Severity	ID	Finding
CRITICAL	B-C01	<code>AtomicU64</code> <code>fetch_sub</code> wraps on underflow. If current balance is \$50 and we subtract \$100, the result becomes ~18.4 quintillion dollars. The bot then fires massive phantom orders.
CRITICAL	B-C02	TOCTOU race in rebalancer. Balance check and debit are separated by ~60s of blockchain wait. Concurrent trades consume the balance between check and debit.
CRITICAL	B-C03	Deposit verification tolerance is \$1,000,000 instead of \$1. Stale large deposits from days ago can be incorrectly matched, crediting phantom capital.

5. Data Feed and Orderbook

The data feed subsystem provides market prices to the strategy engine via WebSocket connections to 17 exchanges. The ticker feed (datafeed.rs) is significantly more mature than the L2 depth feed (order_book.rs). The most critical issues involve stale data being served after WebSocket disconnections and sequence gaps being detected but not acted upon, which directly causes trading on non-existent liquidity.

5.1 Critical Findings

Severity	ID	Finding
CRITICAL	D-C01	L2 order books are NOT invalidated on WebSocket disconnect/reconnect. After any WS disconnect, L2 books retain prices from before the disconnect. Strategies trade against phantom liquidity.
CRITICAL	D-C02	Sequence gaps are detected but deltas are applied anyway. The book enters an unrecoverable stale state with missing updates. No REST snapshot is requested.
CRITICAL	D-C03	WebSocket Close frames permanently kill feed workers. Exchange WS servers routinely send Close frames during maintenance. After this, exchange prices are permanently frozen.

5.2 High Findings

Severity	ID	Finding
HIGH	D-H01	Ticker feed parser key collision: scanning for single-byte keys like "t", "b", "a" can match inside JSON string values, producing garbage prices.
HIGH	D-H02	zero_copy_parser.rs uses f64 for prices. Subtle precision loss for low-priced altcoins when compared with fixed-point values from datafeed.rs.
HIGH	D-H05	L2OrderBookListener uses .first() for all symbol writes. All L2 updates for all symbols get merged into one book, corrupting it completely.

6. Exchange Implementations

All 17 exchange implementations were audited line-by-line (~6,500 lines). Two exchanges are completely broken (Bybit and Bitstamp), one has silent error handling failures (OKX), and several have partial fill data loss on order cancellation. Positive findings include excellent nonce management (Bitfinex, Kraken), proper credential handling with SecretString, TLS pinning infrastructure, and consistent 429 backoff across all exchanges.

6.1 Critical Findings

Severity	ID	Finding
CRITICAL	X-C01	Bitstamp HMAC signature preimage has url_path and content_type SWAPPED. ALL Bitstamp authenticated requests (orders, cancels, balance) will fail signature verification.
CRITICAL	X-C02	Bybit sends X-BAPI-SIGN-TYPE: "2" (RSA) header but produces HMAC-SHA256 signature. ALL Bybit V5 API requests will fail authentication.
CRITICAL	X-C03	OKX handle_response only checks HTTP status. OKX V5 returns HTTP 200 with error body for auth/business errors. Silent failures with garbage data parsed as order IDs.
CRITICAL	X-C04	Coinbase cancel_order hardcodes filled_qty=0 without fetching actual fill state. Partial fills before cancellation are lost.
CRITICAL	X-C05	Ibank cancel_order has same partial fill data loss. No fetch_order_status call after cancellation.

Severity	ID	Finding
CRITICAL	X-C06	Deribit sends API secret in JSON-RPC body. Captured by any WAF/proxy/logging layer. Key compromise = full account takeover.

6.2 Exchange Status Matrix

Exchange	Status	Critical	High	Notes
Binance	Viable	0	1	f64 in balance parse; fix needed
Bybit	BROKEN	1	2	SIGN-TYPE header + status mapping
OKX	Risky	1	2	No body code check; double throttle
Coinbase	Partial	0	1	Cancel fill data lost
Bitstamp	BROKEN	1	0	Signature field order swapped
Gate.io	Viable	0	1	Status mapping needs fix
HTX	Caution	0	1	Missing APIKEY header on cancel
Bitget	Good	0	0	Proper pre-computed passphrase
Kraken	Excellent	0	0	Best implementation in codebase
Bitfinex	Excellent	0	0	Monotonic nonce, dual-order source

7. Infrastructure and Support

Infrastructure modules cover rate limiting, persistence, signing, nonce management, configuration, shared memory, health monitoring, metrics, timestamp synchronization, and CPU pinning. The most critical infrastructure finding is that API nonces are not persisted to disk, meaning every restart risks a trading blackout until nonce synchronization succeeds. The persistence layer also lacks fsync before file rename, risking data loss on power failure.

7.1 Critical Findings

Severity	ID	Finding
CRITICAL	I-C01	Nonce not persisted. In-memory AtomicU64 counters reset on restart. Every restart risks complete trading blackout if nonce sync fails.
CRITICAL	I-C02	EXDEV fallback in persistence copies then removes (not atomic). Crash mid-copy corrupts state file. Session PnL and trade counts silently zeroed.
CRITICAL	I-C03	No fsync before rename in persistence. Power loss leaves file metadata pointing to unwritten blocks. Same data loss as I-C02.
CRITICAL	I-C04	Backward clock jump (NTP step, VM migration) not detected. Adjusted timestamp jumps backward, causing all exchange orders to be rejected.

7.2 High Findings

Severity	ID	Finding
HIGH	I-H01	zero_alloc_signer stores API secret as plain String with no Drop impl to zero memory. Secret persists in heap after signer is dropped.
HIGH	I-H02	ValidatedExchangeConfig stores api_key and api_secret as plain Strings. Secrets live in heap for entire process lifetime.

Severity	ID	Finding
HIGH	I-H04	Size slicer ignores exchange max_quantity. For low-priced assets, sliced orders exceed exchange limits and are rejected.
HIGH	I-H05	Dynamic fee refresh task is detached (JoinHandle dropped). If the task panics, fee data goes permanently stale with no recovery.
HIGH	I-H09	Persistence has no integrity checksum. Corrupted state file silently loads as default (zeroed PnL, cleared exchange health).

8. P&L; Reporting, Withdrawal, and Auxiliary Systems

This section covers the P&L; reporter, live order tracker, withdrawal executor, Discord alerts, paper trading, backtesting, subaccount verification, coin finder, triangular path finder, and stablecoin monitoring. The withdrawal system is the most dangerous: it has no address whitelist, no amount validation, no balance check, and no confirmation step. The backtesting module has look-ahead bias and phantom sells, making all backtest results unreliable.

8.1 Critical Findings

Severity	ID	Finding
CRITICAL	P-C01	Withdrawal executor has NO address whitelist validation. Any code path can withdraw to any on-chain address. Direct fund-theft vector.
CRITICAL	P-C02	Withdrawal executor has NO amount validation or balance check. A logic bug could drain the entire exchange balance in a single withdrawal.
CRITICAL	P-C03	Withdrawal has NO confirmation/dry-run mechanism. No safety net between "decide to withdraw" and "funds leave the exchange."
CRITICAL	P-C04	Backtest has intra-timestamp look-ahead bias. Strategy 2 sees Strategy 1 balance changes at the same timestamp. All backtest results unreliable.
CRITICAL	P-C05	Backtest sells phantom inventory. Only USD balances tracked per exchange. Cross-exchange arb sells assets that do not exist on the selling exchange.
CRITICAL	P-C06	Stablecoin upper depeg threshold is broken. $1.0 + 0.998 = 1.998$ instead of 1.002. Upward depegs (e.g., USDT spiking to \$1.05) are NOT detected.

9. Top 10 Priority Fixes

These are the fixes that must be implemented before any live capital is at risk, ordered by impact and effort. Each fix addresses one or more critical findings. The estimated effort is based on the complexity of the change and the number of files affected.

#	Fix	Findings Addressed	Effort
1	Wire SafetyExecutionEngine + CoreExecutionShield into production path	X-01, SE-01, CS-01	4-8h
2	Implement position tracking with balance deduction after fills	E-01, X-02	4-6h
3	Fix asymmetric fill detection (check quantity mismatch, not just XOR)	E-03, CE-01	1-2h
4	Fix AtomicU64 fetch_sub underflow (use fetch_update with saturating_sub)	B-C01	30min

#	Fix	Findings Addressed	Effort
5	Fix deposit verification tolerance (\$1M to \$1)	B-C03	5min
6	Disable triangular arb or integrate tri_path_finder	M-01, S-01	2-4h
7	Fix force_live_mode to hard-fail on exchange init errors	M-02	30min
8	Fix Bybit SIGN-TYPE and Bitstamp signature field order	X-C02, X-C01	15min
9	Invalidate L2 books on WS disconnect; handle Close frames	D-C01, D-C03	1h
10	Remove circuit breaker auto-recovery for NETWORK_PARTITION	R-C05	15min

10. Architectural Gap Analysis

Beyond individual bugs, the audit identified several architectural gaps where the system lacks capabilities that a production HFT system would require. These gaps represent missing functionality rather than broken functionality, and addressing them requires design decisions about risk tolerance and operational procedures.

Gap	Severity	Description
No per-exchange position limit	CRITICAL	The gatekeeper has max total exposure (50%) and max single position (5%) but no per-exchange cap. All capital could be concentrated on one exchange. If that exchange is hacked or goes bankrupt, 100% of capital is at risk.
No per-pair loss limits	HIGH	Losses are tracked in aggregate. A single pair (e.g., a pegged asset that depegs) could accumulate unlimited losses without triggering per-pair limits.
No max concurrent open positions	HIGH	No limit on how many distinct positions can be open simultaneously. Under extreme conditions, hundreds of micro-positions could consume capital and API rate limits.
No rapid-loss circuit breaker	MEDIUM	No protection against e.g., "5 losses in 10 seconds" pause all trading. The system keeps trading as fast as it can even during a sustained losing streak.
No pre-trade balance verification	MEDIUM	The system does not verify actual exchange balances match expected balances before placing orders. A withdrawal or forced liquidation could leave the system trading on non-existent balances.
No external watchdog kill switch	MEDIUM	The kill switch is irreversible and process-internal. There is no mechanism for an external watchdog (separate process, heartbeat monitor) to trigger it.
No structured audit logging	LOW	Risk checks produce tracing logs but no structured audit events. For regulatory compliance, a trade-level risk decision log is needed.
No session PnL reconciliation	LOW	Session PnL is only incremented by record_trade_pnl. There is no reconciliation against actual exchange settlement. If a trade fails after being recorded, the PnL is wrong.

11. Conclusion

This audit identified 35 critical, 66 high, 90+ medium, and 40+ low severity findings across all 77 source files of the rust-hft-arb trading system. The system is not production-ready for live trading with real capital. The most urgent issues fall into four categories: dead safety code that is never executed (3 modules exist but are not wired in), missing balance/position tracking in live mode,

broken exchange integrations (Bybit and Bitstamp are completely non-functional), and a dangerous withdrawal system with no safety controls.

The Top 10 Priority Fixes section provides a concrete remediation roadmap. The highest-impact fix is wiring `SafetyExecutionEngine` and `CoreExecutionShield` into the production execution path, which would immediately add pre-trade price validation, circuit breaker integration, and slippage enforcement. The fastest fixes with highest impact are the deposit verification tolerance bug (\$1M to \$1), the Bybit/Bitstamp authentication fixes, and removing circuit breaker auto-recovery for network partitions. Implementing the Top 10 fixes would address the majority of capital-loss vectors and bring the system significantly closer to production readiness, though additional work on balance tracking, exchange error handling, and data feed resilience would still be required.

The codebase demonstrates strong architectural intent with well-designed protection layers, proper use of `rust_decimal` for all financial calculations, good credential handling with `SecretString`, and a comprehensive exchange abstraction layer. The issues found are primarily integration gaps (safety modules not wired in), missing edge-case handling, and a few critical typos in exchange-specific authentication code. With focused remediation on the identified priorities, the system has a solid foundation to build upon.